Design and Implementation of a 32-bit Single-Cycle RISC-V Processor in Verilog HDL

Maimoona Saboor
Reg: 2024270

Labiba Ahmad
Reg: 2024260

***Abstract*—This paper presents the design, implementation, and simulation of a 32-bit single-cycle RISC-V processor using Verilog HDL. The processor supports a subset of the RV32I instruction set, including arithmetic, logical, memory access, and conditional branch instructions. The architecture integrates key components such as the program counter, instruction memory, register file, arithmetic logic unit (ALU), control unit, and data memory. Functional verification is performed using simulation testbenches and waveform analysis, confirming correct instruction execution and control flow behavior.**

I. Introduction

RISC-V is an open-source instruction set architecture (ISA) that has gained significant popularity in both academia and industry due to its modularity, simplicity, and extensibility. Unlike proprietary ISAs, RISC-V allows designers to implement processors tailored for specific applications without licensing restrictions.

This project focuses on designing a single-cycle 32-bit RISC-V processor using Verilog HDL. In a single-cycle architecture, each instruction is executed completely within one clock cycle, making the design conceptually simple and ideal for educational purposes.

A. Verilog Implementation

The complete Verilog HDL implementation of the processor, including all modules, testbenches, and simulation files, can be accessed through the following link:

Code Repository Github

B. Problem Statement

The objective of this project is to design and implement a functional 32-bit single-cycle RISC-V processor capable of executing a subset of instructions while maintaining correctness and simplicity of design. The processor is required to:

1) Accurately fetch, decode, and execute RISC-V instructions in a single clock cycle, ensuring proper sequencing through the instruction lifecycle.
2) Support a core subset of the RISC-V ISA, including arithmetic (add, sub), logical (and, or), immediate (addi), memory access (lw, sw), and control flow (beq) instructions.
3) Generate precise control signals through a centralized control unit to coordinate datapath components such as the ALU, register file, and memory modules.
4) Ensure correct data propagation across all stages of the datapath, including proper selection of ALU operands, write-back sources, and memory interfaces.
5) Implement reliable conditional branch execution by evaluating comparison results using the ALU and updating the program counter (PC) based on branch conditions.
6) Maintain architectural constraints such as word alignment in memory access, correct handling of immediate values through sign extension, and preservation of register x0 as a constant zero register.
7) Achieve functional correctness through simulation by validating instruction execution, register updates, memory operations, and control flow behavior.

II. Methodology

A. System Architecture

The processor follows a single-cycle datapath architecture in which all operations are completed within one clock cycle. Combinational logic is used for execution, while sequential elements such as the program counter and register file store state information.

B. Instruction Execution Flow

Each instruction in the processor progresses through five conceptual stages that collectively define the instruction execution lifecycle. Although the architecture is single-cycle, these stages represent logical partitions of functionality within the datapath:

- **Instruction Fetch (IF):** The Program Counter (PC) provides the address of the current instruction, which is fetched from instruction memory. Simultaneously, the next sequential address (PC + 4) is computed.
- **Instruction Decode (ID):** The fetched instruction is decoded to extract opcode, source and destination register indices, and function fields. The control unit generates the appropriate control signals, and the register file outputs the required operand values.
- **Execute (EX):** The Arithmetic Logic Unit (ALU) performs the required operation based on the decoded instruction. This includes arithmetic, logical operations, and address calculation for memory instructions. For branch instructions, comparison is performed to determine control flow decisions.
- **Memory Access (MEM):** For load and store instructions, data memory is accessed using the address computed

by the ALU. Read or write operations are performed depending on the control signals.

- **Write Back (WB):** The final result, either from the ALU or memory, is written back to the destination register in the register file, completing the instruction execution.

Although these stages resemble those of a pipelined processor, they are executed sequentially within a single clock cycle. As a result, all combinational operations must complete within one cycle, making the clock period dependent on the longest propagation delay in the datapath.

C. Datapath Design

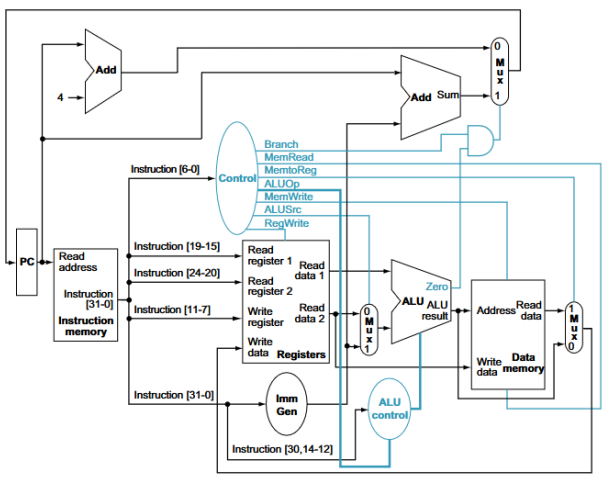

Fig. 1. Complete Single-Cycle RISC-V Datapath

The datapath integrates all major components including the ALU, register file, instruction memory, and data memory, connected through multiplexers and control signals.

D. Key Components

1) Program Counter (PC): The Program Counter holds the address of the current instruction and updates every cycle based on sequential execution or branch decisions.

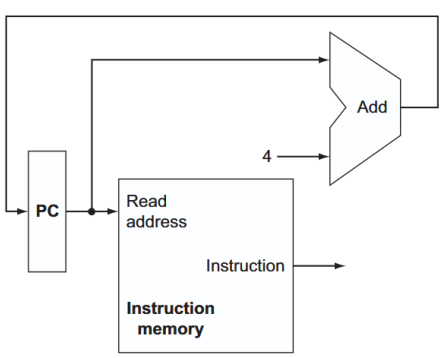

Fig. 2. Program Counter Logic

2) Instruction Memory: Instruction memory stores program instructions in little-endian format and outputs a 32-bit instruction based on the PC address.

3) Register File: The register file consists of 32 general-purpose registers with two read ports and one write port. Register x0 is hardwired to zero.

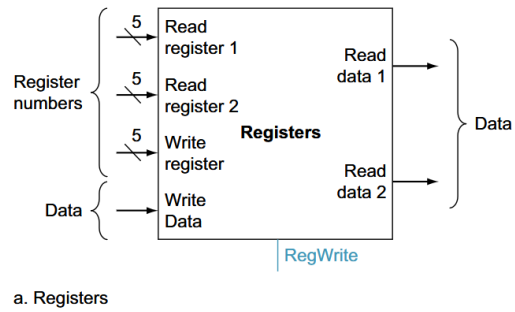

Fig. 3. Register File Architecture

4) Control Unit: The control unit decodes opcode fields and generates control signals to coordinate datapath operations.

TABLE I
CONTROL SIGNALS AND THEIR FUNCTIONS

Signal	Function
ALUSrc	Select ALU input source
MemtoReg	Select write-back source
RegWrite	Enable register write
MemRead	Enable memory read
MemWrite	Enable memory write
Branch	Enable branch decision
ALUOp	Defines ALU operation type

5) Arithmetic Logic Unit (ALU): The ALU performs arithmetic and logical operations including ADD, SUB, AND, OR, and comparison operations. It also generates a zero flag used for branch decisions.

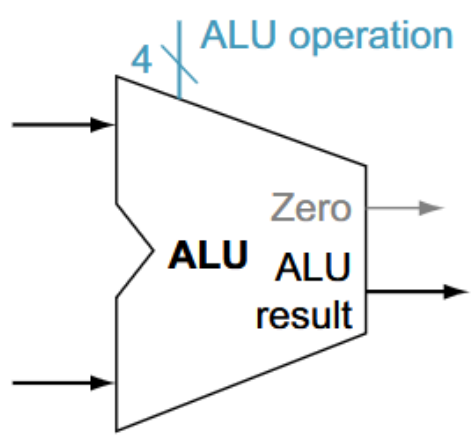

Fig. 4. ALU Design and Operation

6) Data Memory: Data memory supports word-aligned read and write operations used by load and store instructions.

E. Branch Control Logic

Branch decisions are made using the following logic:

$$PC_{src} = Branch \wedge Zero \tag{1}$$

If the condition is true, the program counter is updated with the branch target address.

III. Instruction Type Implementation

The processor implements a subset of the RV32I instruction set architecture. Each instruction type not only differs in format but also in how data flows through the datapath and how control signals are generated to coordinate execution.

A. R-Type Instructions

R-type instructions perform register-to-register operations and are fully executed within the ALU without involving memory.

$$\text{R-type: } [funct7 \mid rs2 \mid rs1 \mid funct3 \mid rd \mid opcode] \tag{2}$$

Supported Instructions: add, sub, and, or
Detailed Execution:

- The register file reads two operands simultaneously from rs1 and rs2
- The ALU control unit decodes funct3 and funct7 to determine the exact operation
- The ALU performs the computation in a purely combinational manner
- The result is routed directly back to the register file through the write-back path

Key Insight: No memory access or immediate generation is required, making this the most direct datapath usage.
Control Signals:

- ALUSrc = 0 (second operand from register)
- RegWrite = 1 (write result to rd)
- MemRead = 0, MemWrite = 0
- MemtoReg = 0 (ALU result selected)
- ALUOp = 10 (function-based decoding)

B. I-Type Instructions (Immediate)

I-type instructions introduce immediate values, enabling operations with constants.

$$\text{I-type: } [imm[11:0] \mid rs1 \mid funct3 \mid rd \mid opcode] \tag{3}$$

Supported Instruction: addi
Detailed Execution:

- The immediate field is extracted and sign-extended to 32 bits
- One operand is read from rs1, while the second comes from the immediate generator
- A multiplexer selects the immediate instead of rs2 as the ALU input
- The ALU performs the operation and forwards the result to rd

Key Insight: The introduction of the immediate path requires additional hardware (sign extender + mux), increasing datapath flexibility.
Control Signals:

- ALUSrc = 1 (select immediate)
- RegWrite = 1
- MemtoReg = 0
- ALUOp = 10

C. Load Instructions (lw)

Load instructions involve both computation and memory access.

$$\text{Load: } [imm[11:0] \mid rs1 \mid funct3 \mid rd \mid opcode] \tag{4}$$

Detailed Execution:

- The effective memory address is computed by the ALU as rs1 + immediate
- Data memory is accessed using this computed address
- The retrieved data is passed through a multiplexer to the register file
- The value is written into rd during the write-back stage

Key Insight: This instruction utilizes both the ALU (for address calculation) and memory, increasing critical path length.
Control Signals:

- ALUSrc = 1
- MemRead = 1
- MemtoReg = 1 (select memory output)
- RegWrite = 1
- ALUOp = 00 (addition)

D. Store Instructions (sw)

Store instructions transfer data from registers to memory without writing back to the register file.

$$\text{Store: } [imm[11:5] \mid rs2 \mid rs1 \mid funct3 \mid imm[4:0] \mid opcode] \tag{5}$$

Detailed Execution:

- The ALU computes the effective address using rs1 and the sign-extended immediate
- The value from rs2 is forwarded directly to data memory
- Memory write is enabled, storing the data at the computed address

Key Insight: Unlike load instructions, no data returns to the register file, eliminating the write-back stage for this instruction.
Control Signals:

- ALUSrc = 1
- MemWrite = 1
- RegWrite = 0
- ALUOp = 00

E. B-Type Instructions (Branch)

Branch instructions modify control flow based on a condition.

$$\text{Branch: } [imm \mid rs2 \mid rs1 \mid funct3 \mid imm \mid opcode] \quad (6)$$

Supported Instruction: beq
Detailed Execution:

- The register file provides operands rs1 and rs2
- The ALU performs subtraction to check equality
- A zero flag is generated if the operands are equal
- The immediate is sign-extended and shifted to form a branch offset
- If the branch condition is satisfied, the PC is updated to PC + offset; otherwise, PC + 4 is used

Key Insight: Branch instructions introduce control hazards conceptually, even in single-cycle designs, as they determine the next instruction dynamically.

Control Signals:

- ALUSrc = 0
- Branch = 1
- RegWrite = 0
- ALUOp = 01 (subtraction for comparison)

IV. Results and Simulation

A. Instruction Verification

The processor was tested using a custom Verilog testbench.

TABLE II
INSTRUCTION EXECUTION RESULTS

Instruction	Expected Result	Observed Result
addi	10	Correct
add	15	Correct
sub	5	Correct
and	0	Correct
or	15	Correct
lw/sw	Memory Access	Correct

B. Branch Behavior

- Branch not taken: PC increments normally
- Branch taken: PC updates to target address correctly

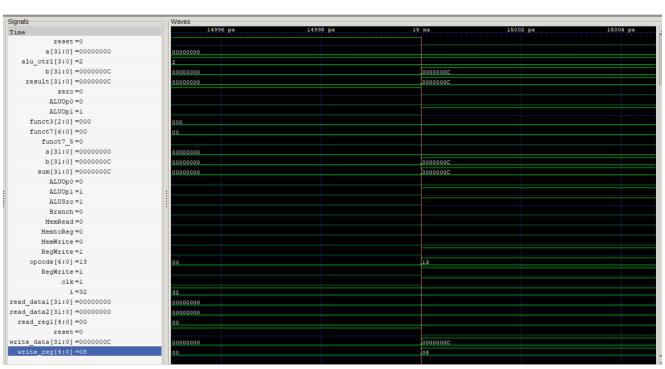

Fig. 5. Output Waveform

```
              REGISTER FILE RESULTS
-------------------------------------------
x16 = 10 | expected = 10
x17 =  5 | expected =  5
x18 = 20 | expected = 20
x19 =  3 | expected =  3
x5  = 15 | expected = 15
x6  =  5 | expected =  5
x7  =  0 | expected =  0
x28 = 15 | expected = 15
x29 = 23 | expected = 23
x30 = 17 | expected = 17
x31 = 15 | expected = 15
-------------------------------------------
              DATA MEMORY RESULTS
-------------------------------------------
dmem[0]  = 15 | expected = 15
-------------------------------------------
                  PC RESULT
-------------------------------------------
PC       = 52 | expected = 52  when exiting
```

Fig. 6. Terminal Outputs Showing Instruction Execution

```
-------------------------------------------
                  PC RESULT
-------------------------------------------
PC       = 52 | expected = 52  when exiting
time=195000 PC=        48
time=205000 PC=        52
time=215000 PC=        48
time=225000 PC=        52
time=235000 PC=        48
time=245000 PC=        52
time=255000 PC=        48
time=265000 PC=        52
time=275000 PC=        48
time=285000 PC=        52
testbench.v:74: $finish called at 290000 (1ps)
```

Fig. 7. PC upgrades showing branching and increments

C. Program Counter Analysis

The program counter correctly updates for sequential execution and branching scenarios, confirming proper control flow implementation.

V. Conclusion

This project presents the successful design and implementation of a 32-bit single-cycle RISC-V processor using Verilog HDL. The processor is capable of correctly executing a selected subset of the RV32I instruction set, including arithmetic, logical, memory access, and branch instructions.

The implementation demonstrates correct integration of key architectural components such as the datapath, control unit, ALU, register file, instruction memory, and data memory. Through simulation and waveform analysis, the processor was verified to perform accurate instruction fetch, decode, execute, memory access, and write-back operations within a single clock cycle.

Overall, the project highlights the practical understanding of processor design principles, control signal generation, and datapath coordination in a single-cycle architecture, while also reinforcing the trade-offs between simplicity and performance inherent in such designs.

A. Future Work

Future improvements include:

- Implementation of pipelined architecture for higher performance
- Introduction of hazard detection and forwarding units
- Expansion of instruction set (RV32M extensions)
- Integration of cache memory for performance optimization

References

[1] RISC-V International, "K. Henney, *Computer Organization and Architecture*, 1st ed. Pearson, 2019.